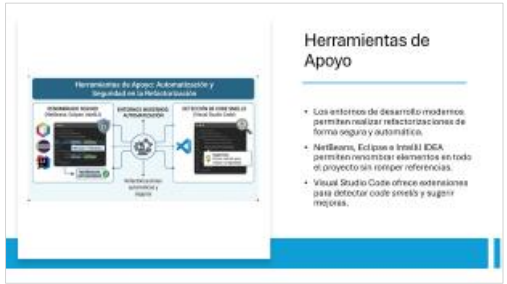
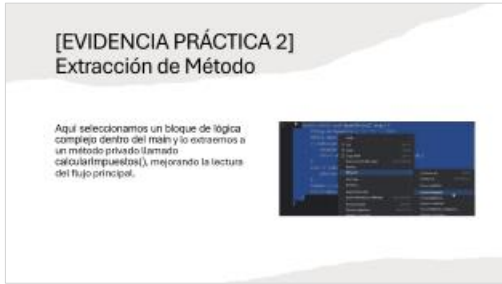
UT4: Optimización y Documentación

Oscar Rodríguez y Andrei Moisei

Introducción a la Refactorización





¿Qué es la Refactorización?

La refactorización es una técnica de ingeniería de software para reestructurar un código existente, alterando su estructura interna sin cambiar su comportamiento externo. Su objetivo es limpiar el código "sucio" para hacerlo más comprensible y barato de modificar.

Beneficios:

- La legibilidad facilita que otros desarrolladores entiendan la lógica rápidamente.
- La mantenibilidad reduce la deuda técnica y facilita futuras actualizaciones.
- Calidad del diseño: un código más limpio es menos propenso a errores ocultos.

Limitaciones y Riesgos

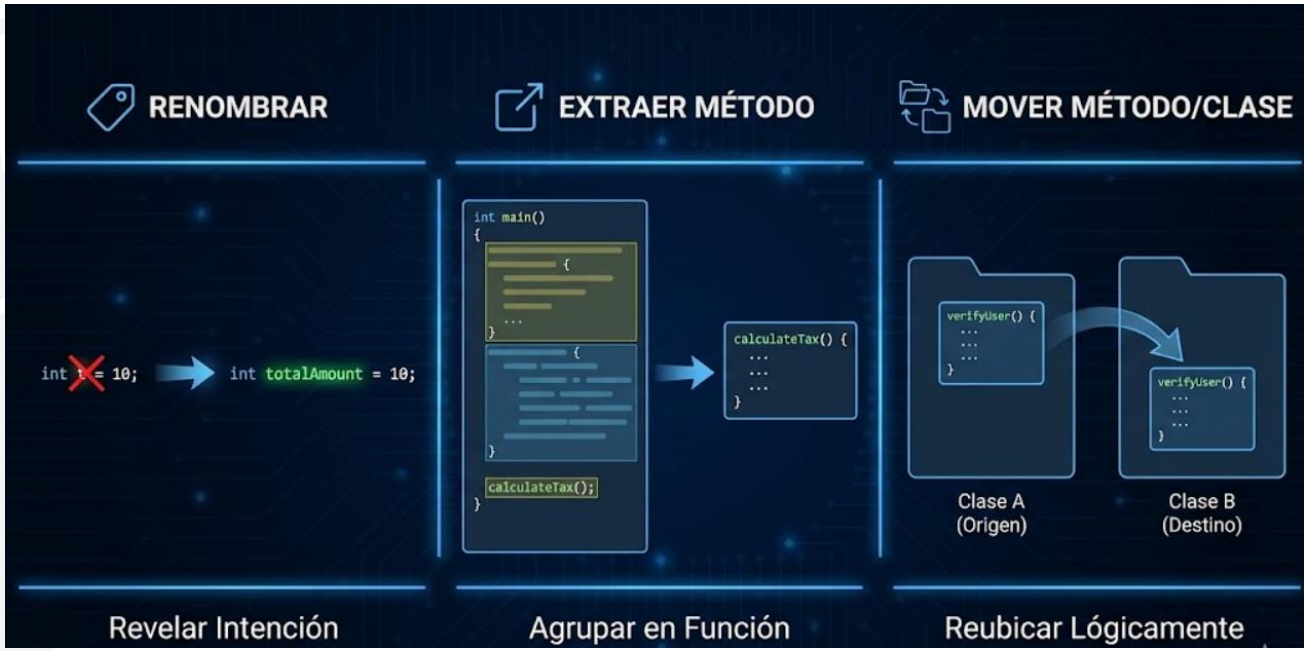
- No añade nuevas funcionalidades al sistema.
- No realiza corrección de errores (Debugging), aunque ayuda a encontrarlos, su fin no es arreglar fallos.
- No arregla malos diseños profundos. Si la base es errónea, refactorizar pequeñas partes no salvará el proyecto.
- La refactorización no corrige problemas profundos de arquitectura ni decisiones de diseño incorrectas tomadas en fases iniciales del proyecto.
- Requiere tiempo y esfuerzo que no se traduce en nuevas características visibles para el cliente.

Patrones Básicos (I)

Renombrar: Cambiar nombres de variables, clases o métodos para que revelen su intención.

Extraer Método: Tomar un fragmento de código largo, agruparlo y moverlo a un método separado con un nombre descriptivo.

Mover Método/Clase: Reubicar funcionalidades donde tengan más sentido lógico o arquitectónico.

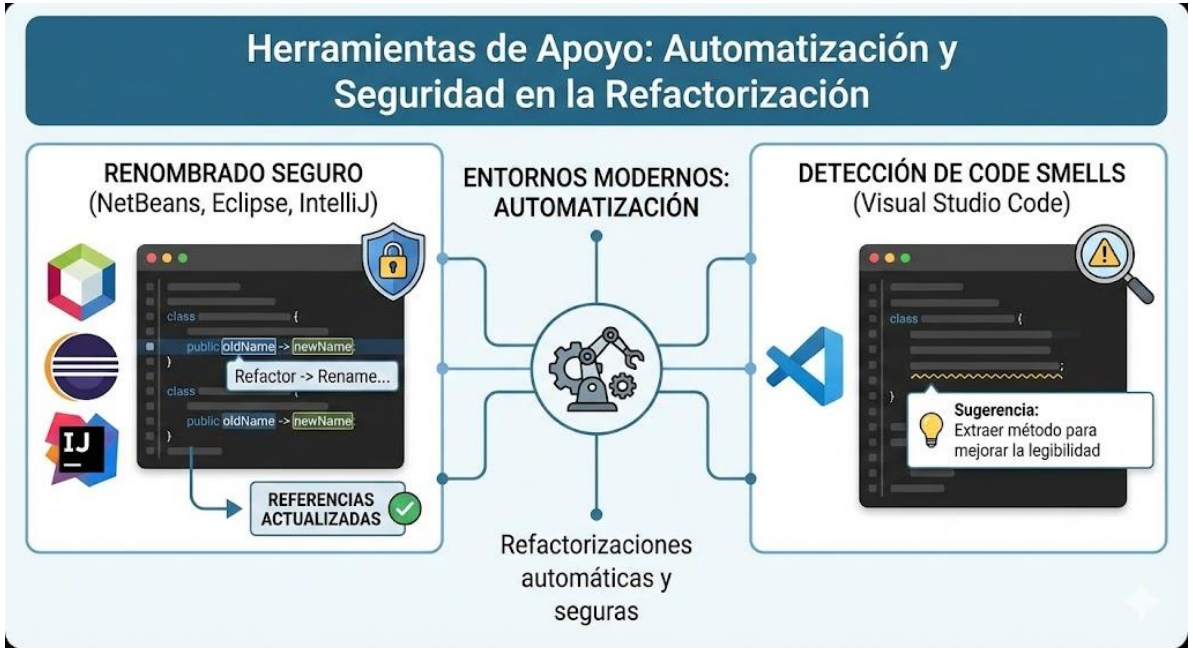


Patrones Básicos (II)

- **Eliminar Código Duplicado**
- **Simplificación de Condicionales:** Reemplazar lógica anidada compleja.
- **Dividir métodos muy largos.**
- **Eliminar parámetros innecesarios:** Limpiar las firmas de los métodos para que solo pidan lo que realmente usan.

Refactorización y Pruebas

- Antes de refactorizar, debes contar con pruebas automatizadas fiables que garanticen que el comportamiento del sistema se mantiene correcto.
- **El Ciclo:** Test (Verde) → Refactorizar → Test (Sigue Verde).
- **Garantía:** Las pruebas aseguran que el comportamiento externo no ha cambiado, cumpliendo la regla de la refactorización.
- Las pruebas unitarias son las más efectivas para validar cambios en métodos o clases pequeñas.

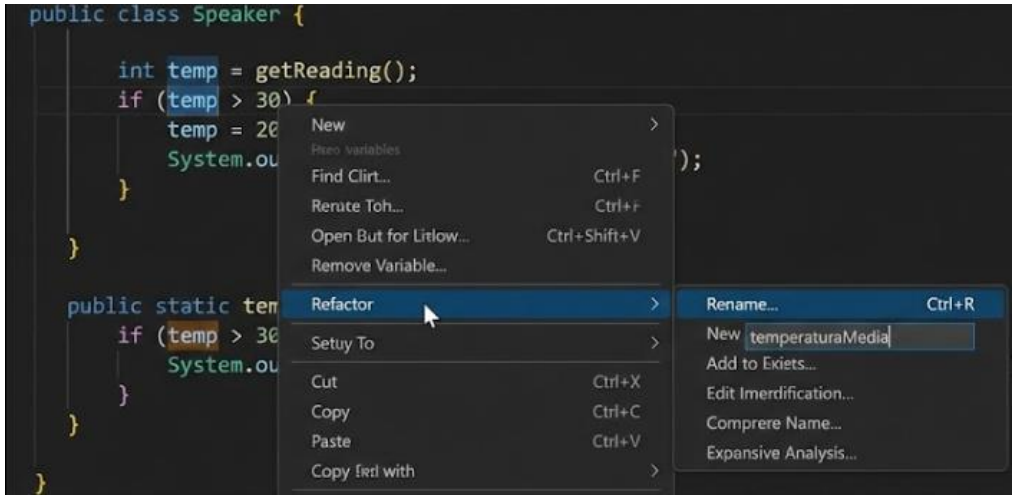


Herramientas de Apoyo

- Los entornos de desarrollo modernos permiten realizar refactorizaciones de forma segura y automática.
- NetBeans, Eclipse e IntelliJ IDEA permiten renombrar elementos en todo el proyecto sin romper referencias.
- Visual Studio Code ofrece extensiones para detectar *code smells* y sugerir mejoras.

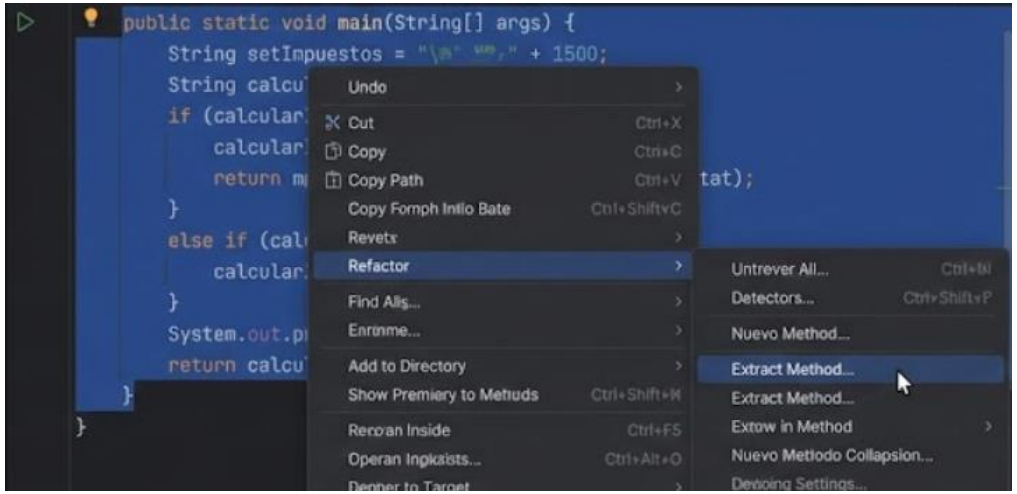
[EVIDENCIA PRÁCTICA 1] Renombrado Seguro

En esta captura vemos cómo el IDE identifica todas las ocurrencias de la variable temp y permite cambiarlas a temperaturaMedia en todo el archivo simultáneamente, evitando errores de tipeo manual.

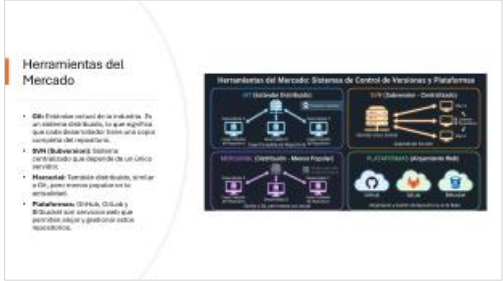


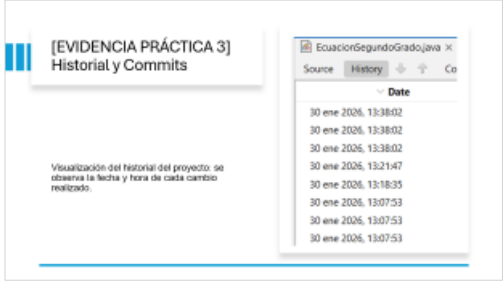
[EVIDENCIA PRÁCTICA 2] Extracción de Método

Aquí seleccionamos un bloque de lógica complejo dentro del main y lo extraemos a un método privado llamado calcularImpuestos(), mejorando la lectura del flujo principal.



CONTROL DE VERSIONES





Trabajo Colaborativo

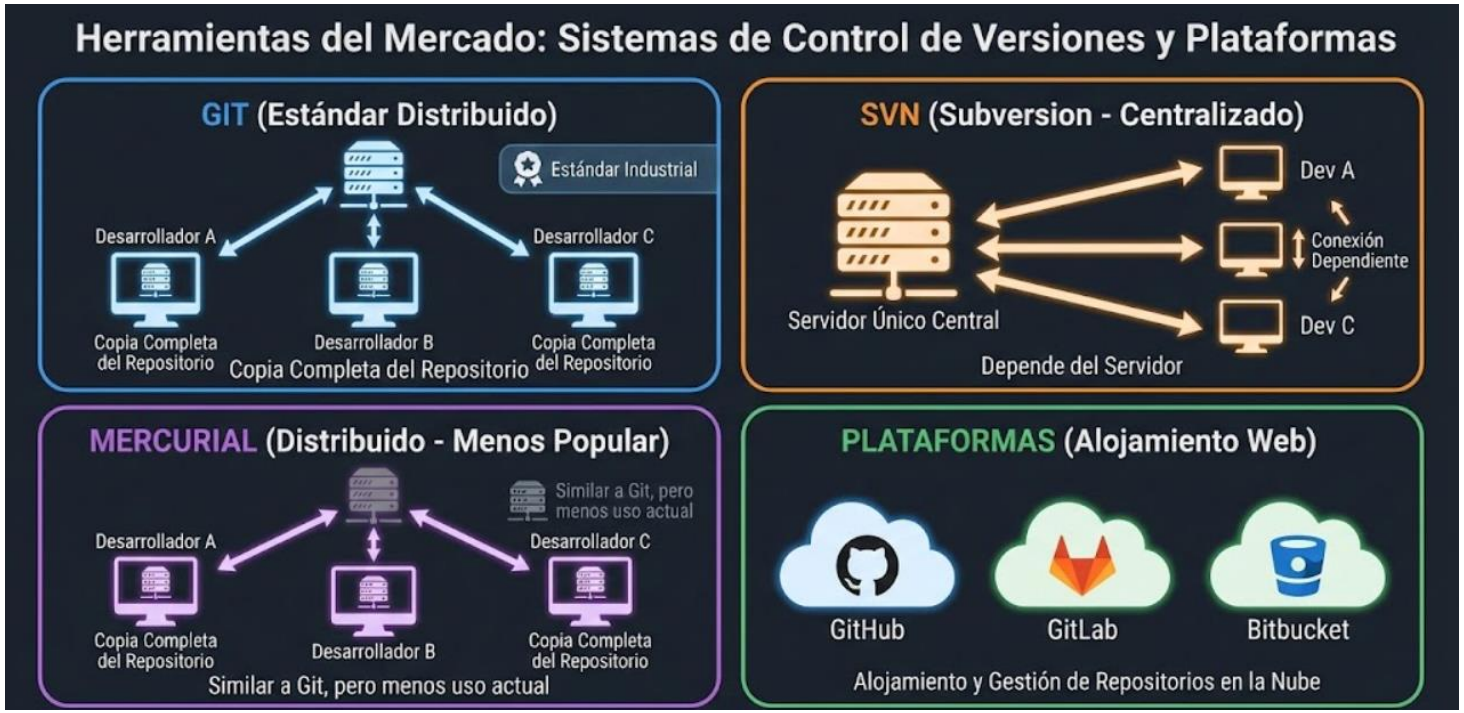
- **Problema:** En entornos de desarrollo colaborativo, varios programadores trabajan simultáneamente sobre los mismos archivos, lo que puede provocar sobrescrituras, pérdidas de cambios y conflictos.

- **Solución:** Utilizar un sistema que registre y gestione cada modificación realizada (quién la hizo, qué se modificó y cuándo), permitiendo un control preciso del historial del proyecto.

- **Sin VCS (Version Control System):** Aparición de múltiples copias de archivos con nombres diferentes, dificultad para identificar la versión correcta y riesgo de perder trabajo realizado.

Herramientas del Mercado

- **Git:** Estándar actual de la industria. Es un sistema distribuido, lo que significa que cada desarrollador tiene una copia completa del repositorio.
- **SVN (Subversion):** Sistema centralizado que depende de un único servidor.
- **Mercurial:** También distribuido, similar a Git, pero menos popular en la actualidad.
- **Plataformas:** GitHub, GitLab y Bitbucket son servicios web que permiten alojar y gestionar estos repositorios.

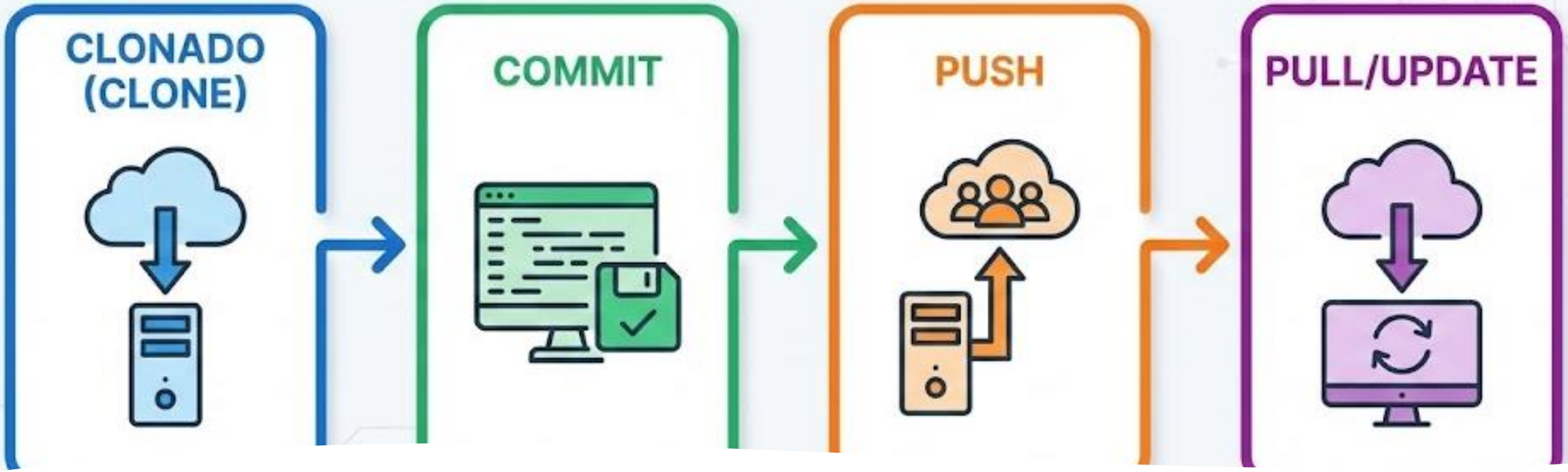


Conceptos Clave

- **Repositorio remoto:** La versión “oficial” alojada en el servidor o en la nube.
- **Copia local:** Tu propia versión de trabajo en el ordenador; permite trabajar sin conexión.
- **Modelo cliente/servidor:** Los clientes (tu PC) sincronizan cambios con el servidor cuando están listos.

Funcionalidades Esenciales

- **Historial:** Permite revisar versiones anteriores del código y su evolución a lo largo del tiempo.
- **Ramas (Branches):** Facilitan el desarrollo paralelo de nuevas funcionalidades sin comprometer la estabilidad de la rama principal (Master/Main).
- **Fusiones (Merge):** Integran los cambios de una rama secundaria en la rama principal de manera controlada.

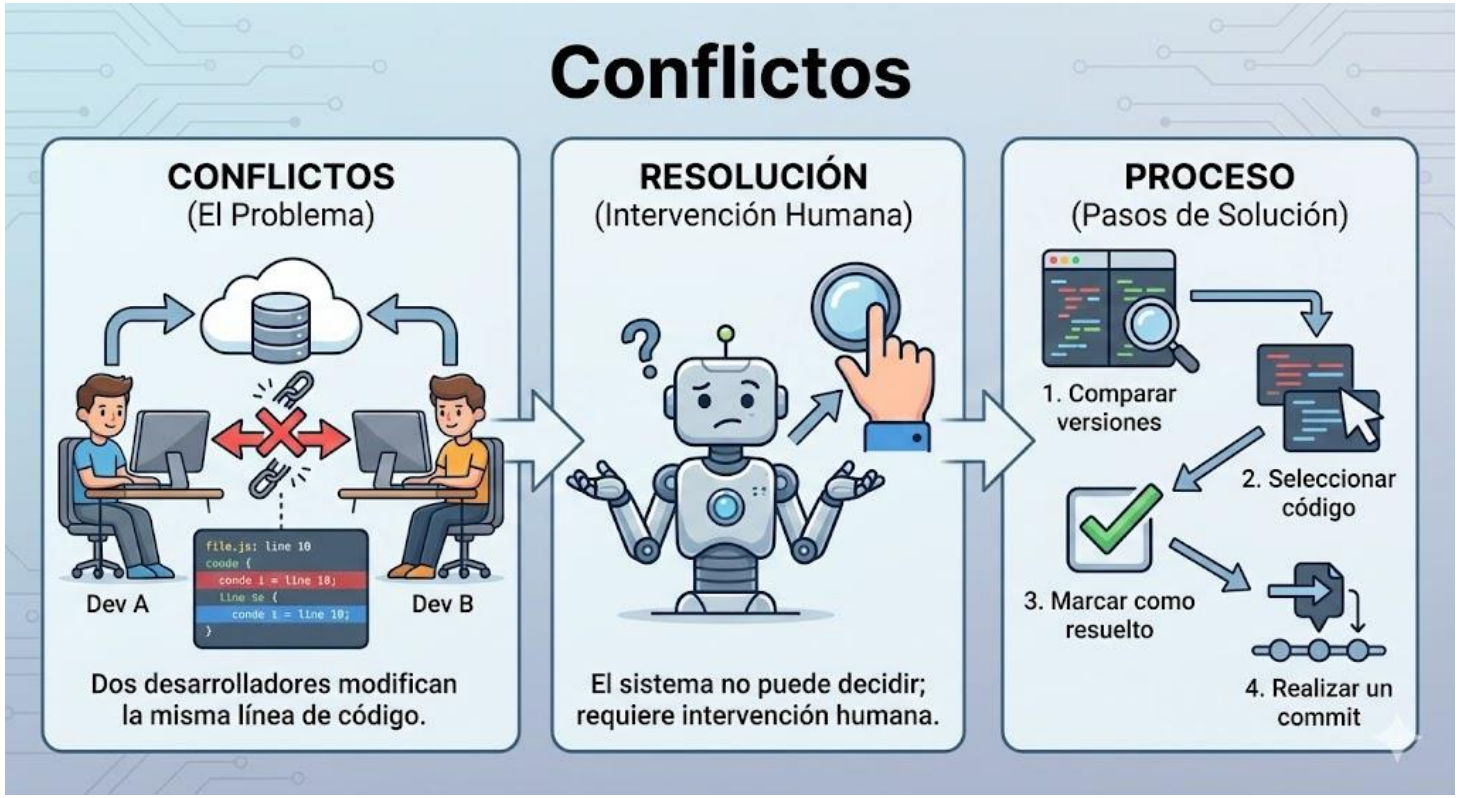


Flujo de Trabajo Diario

- **Clonado (Clone):** Descarga inicial de una copia completa del proyecto.
- **Commit:** Registrar cambios en el historial local, capturando el estado actual del código.
- **Push:** Enviar los commits locales al repositorio remoto para compartirlos con el equipo.
- **Pull/Update:** Actualizar tu copia local con los cambios realizados por otros desarrolladores en el servidor.

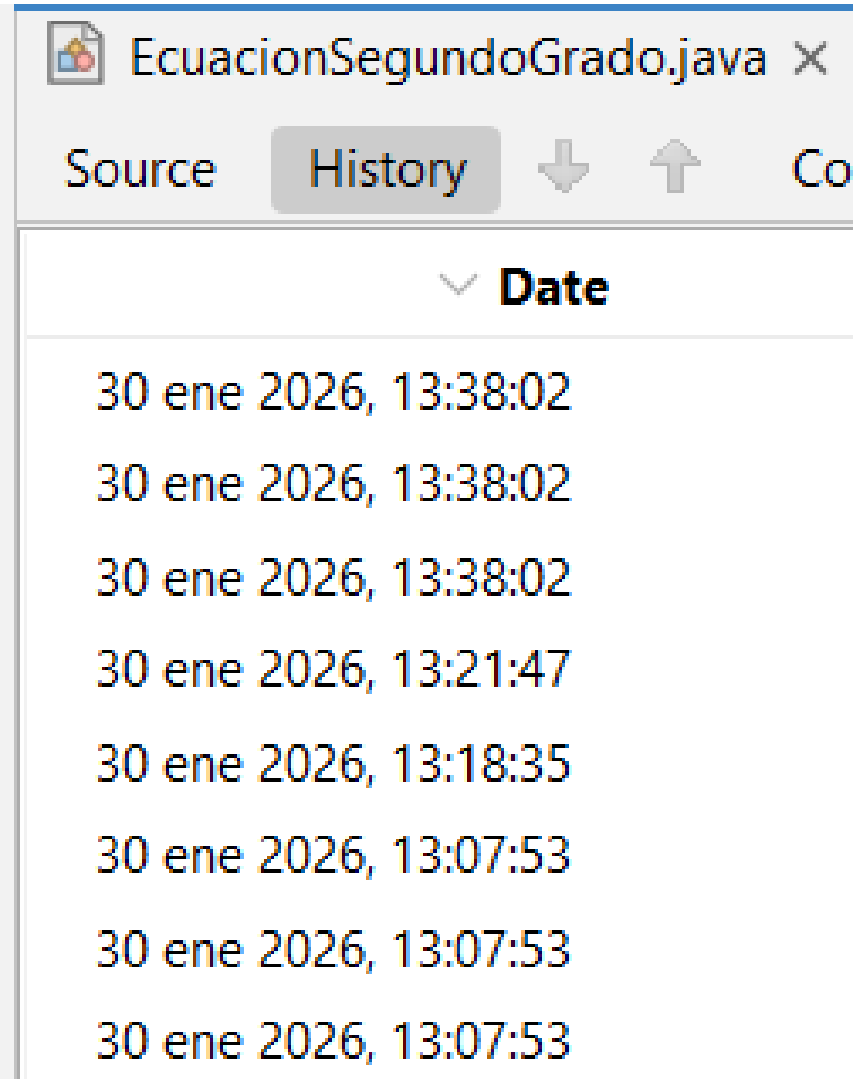
Conflictos

- **Conflictos:** Ocurren cuando dos desarrolladores modifican la misma línea de código y ambos intentan subir sus cambios.
- **Resolución:** El sistema no puede decidir automáticamente; requiere intervención humana.
- **Proceso:** Comparar las versiones en conflicto → seleccionar el código que se conservará → marcar como resuelto → realizar un commit

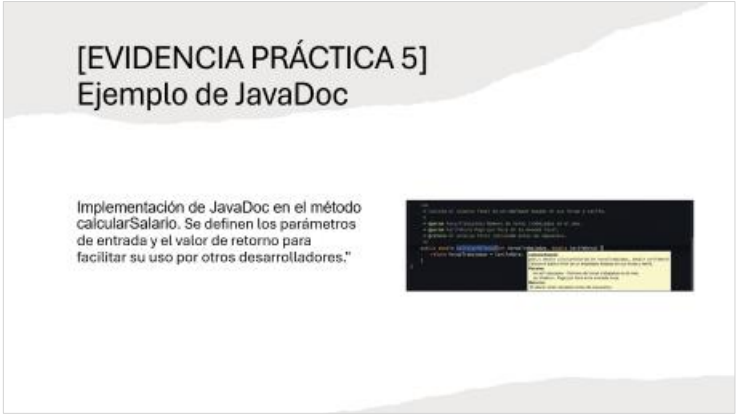


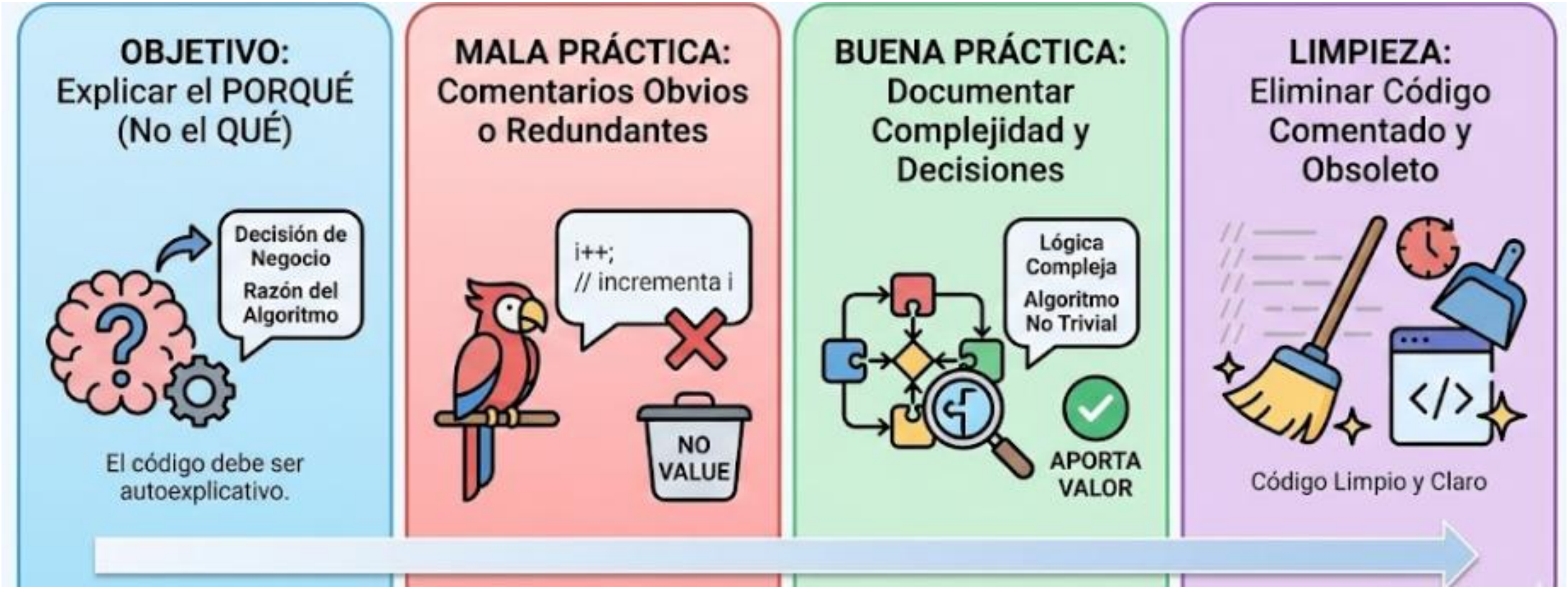
[EVIDENCIA PRÁCTICA 3] Historial y Commits

Visualización del historial del proyecto: se observa la fecha y hora de cada cambio realizado.



DOCUMENTACIÓN



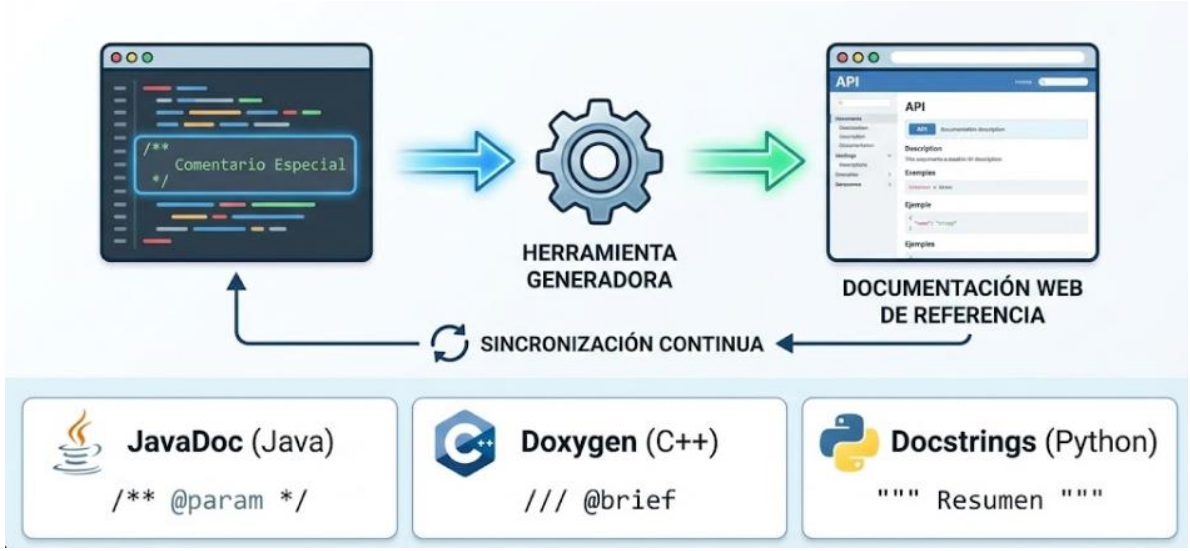


Comentarios en Código

- El objetivo de los comentarios es explicar por qué se hace algo, no qué hace el código; este último debe ser autoexplicativo.
- Un ejemplo de la mala practica es poner comentarios que repiten lo evidente y no aportan valor al entendimiento del código.
- Una buena practica es documentar decisiones complejas de negocio o algoritmos que no se entienden a simple vista.
- La limpieza consiste en eliminar código comentado innecesario y comentarios obsoletos que puedan confundir.

Documentación Automática

Existen herramientas que leen comentarios especiales en el código y generan documentación web de referencia, manteniéndola siempre sincronizada con el desarrollo. Entre las más comunes se encuentran JavaDoc para Java, Doxygen para C++ y Docstrings para Python.



Documentación Externa

- **README.md:** Funciona como la portada del proyecto y debe explicar cómo instalar y usar el software.
- **Wikis:** Se utilizan para guías de estilo, arquitectura y tutoriales extensos.
- **Manuales técnicos:** Dirigidos a administradores o futuros desarrolladores.
- **Diagramas UML:** Representación visual de clases y relaciones que permite comprender la estructura del proyecto de un vistazo.

[EVIDENCIA PRÁCTICA 5]
Ejemplo de JavaDoc

Implementación de JavaDoc en el método calcularSalario. Se definen los parámetros de entrada y el valor de retorno para facilitar su uso por otros desarrolladores."

```java
/**
 * Calcula el salario final de un empleado basado en sus horas y tarifa.
 *
 * @param horasTrabajadas Número de horas trabajadas en el mes.
 * @param tarifaHora Pago por hora en la moneda local.
 * @return El salario total calculado antes de impuestos.
 */
public double calcularSalario(int horasTrabajadas, double tarifaHora) {
    return horasTrabajadas * tarifaHora;
}
}
```

calcularSalario
public double calcularSalario(int horasTrabajadas, double tarifaHora)
Calcula el salario final de un empleado basado en sus horas y tarifa.
Params:
horasTrabajadas - Número de horas trabajadas en el mes.
tarifaHora - Pago por hora en la moneda local.
Returns:
El salario total calculado antes de impuestos.

BANCO DE PREGUNTAS

1. ¿Cuál es el objetivo principal de la refactorización?

- a) Añadir nuevas funciones
- b) Corregir errores graves
- c) Mejorar el código sin cambiar lo que hace ✅
- d) Documentar el sistema

2. ¿Qué sistema de control de versiones es distribuido y el más usado?

a) SVN

b) Git ✅

c) JavaDoc

d) CVS

3. ¿Qué comando sube los cambios locales al repositorio remoto?

a) Pull

b) Clone

c) Merge

d) Push ✅

4. ¿Qué es un repositorio?

a) Carpeta local
b) Servidor de correo
c) Lugar donde se guarda el proyecto ✅
d) Editor de texto

5. ¿Qué herramienta genera documentación automática en Java?

a)Git
b) Doxygen
c) JavaDoc ✅
d) Wiki